```
function
ps_defo_model(Btemp,Nifgs,Npsc,Npsp,Nps,Npsc_selections,std_param,xc0,yc0,zc0,r0
,r10,epoch,defo_method,filenames_output)

% Main script for the subsidence bowl estimation
%
% Input:    - Btemp             temporal baselines
%           - Nifgs             number of interferograms
%           - Npsc              number of psc
%           - Npsp              number of psp
%           - Nps               number of ps
%           - Npsc_selections   number of psc selections
%           - std_param         standard deviations of parameters
%           - xc0               initial range coordinate of the
%                               centre of the bowl
%           - yc0               initial azimuth coordinate of the
%                               centre of the bowl
%           - zc0               initial z coordinate of the centre of the bowl
%           - r0                initial radius of the bowl (size Nepoch)
%           - r10               initial max depth of the bowl (size Nepoch)
%           - epoch             epoch used for bowl estimation
%                               (r and r1 will be estimated for these epochs)
%           - defo_method       deformation model estimation method
%                               0 = kriging
%                               1 = bowl: r1, r, xc, yc, zc
%                               2 = bowl: r1, r, zc
%                               3 = bowl: r1, zc
%           - filenames_output  filenames for output
%
% ------------------------------------------------------------------------
% File.............: ps_defo_model.m
% Version & Date..: 1.7.2.16, 12-DEC-2009
% Authors.........: Astrid Humme
%                   Freek van Leijen
%                   Delft Institute of Earth Observation and Space Systems
%                   Delft University of Technology
% ------------------------------------------------------------------------
%
% This software is developed by Delft University of Technology and is
% intended for scientific use only. Applications for commercial use are
% prohibited.
%
% Copyright (c) 2004-2009 Delft University of Technology, The Netherlands
%
% Change log
%

global project_id Npar_max az_spacing r_spacing m2ph
global ps_eval_method max_mem_buffer fig results_id

% ------------------------------------------------------------------------
% Initialize
% ------------------------------------------------------------------------

Nps_bowl = repmat(NaN,Npsc_selections,1);


% ------------------------------------------------------------------------
% Loop
% ------------------------------------------------------------------------

for z = 1:Npsc_selections

  %----------------------------------------------------------------------
```

```
  % Read data
  %--------------------------------------------------------------------

  ps_fid = fopen([project_id '_ps_results_' results_id '_sel' num2str(z)
'.raw'],'r');
  ps_data = fread(ps_fid,[6+Npar_max+3*Nifgs Nps(z)],'double')';
  fclose(ps_fid);
  %fig = fig+1;
  %figure(fig);hold on
  %for v = 1:Nps(z)
  %  plot(ps_data(v,Npar_max+7:Npar_max+6+Nifgs));
  %end
  %fig = fig+1;
  %figure(fig);hold on
  %for v = 1:Nps(z)
  %  plot(ps_data(v,Nifgs+Npar_max+7:Npar_max+6+2*Nifgs));
  %end
  %fig = fig+1;
  %figure(fig);hold on
  %for v = 1:Nps(z)
  %  plot(ps_data(v,2*Nifgs+Npar_max+7:Npar_max+6+3*Nifgs));
  %end

  ps_H = ps_data(:,4);
  ps_defo1 = ps_data(:,7);
  ps_data = [ps_data(:,1:Npar_max+6)
ps_data(:,Nifgs+Npar_max+7:2*Nifgs+Npar_max+6)];
  %fig = fig+1;
  %  figure(fig);hold on
  %  for v = 1:Nps(z)
  %    plot(ps_data(v,Npar_max+7:end));
  %  end
  %fig = fig+1;

%figure(fig);scatter(ps_data(:,2),ps_data(:,3),20,ps_data(:,Npar_max+7));colorba
r

  %fig = fig+1;
  %figure(fig);scatter(ps_data(:,2),ps_data(:,3),20,ps_data(:,4));colorbar
  %fig = fig+1;
  %figure(fig);scatter(ps_data(:,2),ps_data(:,3),20,ps_data(:,7));colorbar

  %--------------------------------------------------------------------
  % Remove outliers
  %--------------------------------------------------------------------

  ps_H_mean = mean(ps_H);
  ps_H_std = std(ps_H);
  index = find(abs(ps_H-ps_H_mean)<2*ps_H_std);
  ps_data = ps_data(index,:);
  ps_defo1 = ps_defo1(index);

  ps_defo1_mean = mean(ps_defo1);
  ps_defo1_std = std(ps_defo1);
  ps_defo1_diff = ps_defo1-ps_defo1_mean;
  index = find(abs(ps_defo1_diff)>2*ps_defo1_std);

  remove_index = [];
  for v = 1:length(index)
    dist = sqrt(((ps_data(:,2)-ps_data(index(v),2))*az_spacing).^2+...
                ((ps_data(:,3)-ps_data(index(v),3))*r_spacing).^2);
    [dummy,index2] = sort(dist);
    index3 = find(dist(index2(1:10))<100); % only use points <100 m
    if length(index3)>=4
```

```
      index4 = find(abs(ps_defo1_diff(index2(index3))-
ps_defo1_diff(index(v)))<ps_defo1_std);
      if length(index4)<=2 % Do not remove the points with spatial correlation
        remove_index = [remove_index;index(v)];
      end
    end
  end
  ps_data(remove_index,:) = [];
  Nps_bowl(z) = size(ps_data,1);

  %fig = fig+1;
  %figure(fig);scatter(ps_data(:,2),ps_data(:,3),20,ps_data(:,4));colorbar
  %fig = fig+1;
  %figure(fig);scatter(ps_data(:,2),ps_data(:,3),20,ps_data(:,7));colorbar


  % --------------------------------------------------------------
  % Reduce dataset using quadtree decomposition
  % --------------------------------------------------------------

  %if Nps_bowl(z) > 6000
  if Nps_bowl(z) > 1000

    ps_data = ps_quadtree(ps_data);

  end

  %fig = fig+1;
  %figure(fig);scatter(ps_data(:,2),ps_data(:,3),20,ps_data(:,4));colorbar
  %fig = fig+1;
  %figure(fig);scatter(ps_data(:,2),ps_data(:,3),20,ps_data(:,7));colorbar

  %error('dfd')

  % --------------------------------------------------------------
  % Estimation of deformation model
  % --------------------------------------------------------------

  if defo_method == 0

    % ------------------------------------------------------------
    % Estimation by Kriging
    % ------------------------------------------------------------

    ps_defo_kriging(ps_data,Nifgs,Npsc,Npsp,std_param,filenames_output,z);

  else

    % ------------------------------------------------------------
    % Estimation of bowl
    % ------------------------------------------------------------


ps_defo_bowl(ps_data,Nifgs,Npsc,Npsp,Btemp,std_param,xc0,yc0,zc0,r0,r10,epoch,de
fo_method,z);

  end


  % --------------------------------------------------------------------
  % Subtract the modeled deformation
  % --------------------------------------------------------------------

  psc_fid = fopen([project_id '_psc_sel' num2str(z) '.raw'],'r');
```

```
  psc_data = fread(psc_fid,[2*Nifgs+7 Npsc(z)],'double')';

  psc_array = psc_data(:,1:2);
  psc_grid_az = psc_data(:,3);
  psc_grid_r = psc_data(:,4);
  psc_az = psc_data(:,5);
  psc_r = psc_data(:,6);
  psc_phase = psc_data(:,7:Nifgs+6);
  psc_h2ph = psc_data(:,Nifgs+7:2*Nifgs+6);
  psc_amp_disp = psc_data(:,2*Nifgs+7);
  clear psc_data
  fclose(psc_fid);

  psc_fid = fopen([project_id '_psc_5defo_sel' num2str(z) '.raw'],'w');
  psc_defo_fid = fopen([project_id '_psc_defo_sel' num2str(z) '.raw'],'w');
  psc_defo_old = zeros(Npsc(z),Nifgs);
  psc_defo_temp_fid = fopen([project_id '_psc_defo_temp_sel' num2str(z)
'.raw'],'r');
  psc_defo = fread(psc_defo_temp_fid,[Nifgs Npsc(z)],'double')';

  index = find(~isnan(psc_defo(:,1)));
  index2 = find(psc_array(:,2)~=0);
  if ~isempty(setdiff(index-index2,zeros(length(index),1)))
    error('Something is wrong here');
  end

  psc_phase = mod(psc_phase-psc_defo+pi,2*pi)-pi;
  psc_defo = psc_defo_old + psc_defo; %update

  psc_total = [psc_array psc_grid_az psc_grid_r psc_az psc_r psc_phase psc_h2ph
psc_amp_disp];
  fwrite(psc_fid,psc_total','double');
  fclose(psc_fid);

  fwrite(psc_defo_fid,psc_defo','double');
  fclose(psc_defo_fid);


  switch ps_eval_method
    case 'psp'
      Npsp_buffer = floor(max_mem_buffer/(5*Nifgs*8));
      if (Npsp_buffer>=Npsp(1))
        Npsp_buffer = Npsp(1);
        Npsp_rem_buffer = 0;
        Nbuffers = 1;
      else
        Nbuffers = floor(Npsp(1)/Npsp_buffer);
        Npsp_rem_buffer = rem(Npsp(1),Npsp_buffer);
        if (Npsp_rem_buffer > 0)
          Nbuffers = Nbuffers + 1;
        end
      end

      Npsp_sofar = 0;
      psp_defo_temp_fid = repmat(NaN,Npsc_selections,1);
      psp_defo_fid = repmat(NaN,Npsc_selections,1); %write again in
                                                    %correct order
      psp_fid = repmat(NaN,Npsc_selections,1);
      psp_fid_out = repmat(NaN,Npsc_selections,1);

      psp_fid(z) = fopen([project_id '_psp_sel' num2str(z) '.raw'],'r');
      psp_fid_out(z) = fopen([project_id '_psp_5defo_sel' num2str(z)
'.raw'],'w');
      psp_defo_temp_fid(z) = fopen([project_id '_psp_defo_temp_sel' num2str(z)
```

```
'.raw'],'r');
      psp_defo_fid(z) = fopen([project_id '_psp_defo_sel' num2str(z)
'.raw'],'w');

      for v = 1:Nbuffers
        if (v==Nbuffers)&&(Npsp_rem_buffer~=0)
          Npsp_buffer = Npsp_rem_buffer;
        end

        psp_data = fread(psp_fid(z),[2*Nifgs+4 Npsp_buffer],'double')';

        psp_grid_az = psp_data(:,1);
        psp_grid_r = psp_data(:,2);
        psp_az = psp_data(:,3);
        psp_r = psp_data(:,4);
        psp_phase_orig = psp_data(:,5:Nifgs+4);
        psp_h2ph = psp_data(:,Nifgs+5:2*Nifgs+4);
        clear psp_data

        psp_defo = repmat(NaN,Npsp_buffer,Nifgs);
        for w = 1:Nifgs
          fseek(psp_defo_temp_fid(z),(w-1)*8*Npsp(z)+Npsp_sofar*8,-1);
          psp_defo(:,w) = fread(psp_defo_temp_fid(z),[Npsp_buffer 1],'double');
        end
        psp_phase = mod(psp_phase_orig-psp_defo+pi,2*pi)-pi;

        psp_total = [psp_grid_az psp_grid_r psp_az psp_r psp_phase psp_h2ph];
        fwrite(psp_fid_out(z),psp_total','double');

        fwrite(psp_defo_fid(z),psp_defo','double');

      end
      Npsp_sofar = Npsp_sofar+Npsp_buffer;
  end

  fclose('all');
end
```